AI & ML Engineering | End-to-End Machine Learning Pipeline with AWS Glue (Spark)

---

1. Project Overview

This project demonstrates a complete production-style Machine Learning pipeline on AWS using:

- Amazon S3 (data + model storage)
- AWS Glue (Spark-based training & inference)
- Glue Workflow (orchestration)
- IAM (access control)

The pipeline trains a housing price prediction model and performs automated inference.

---

2. Business Problem

Goal: Predict California housing prices using structured housing features.

Dataset: housing.csv

Features:

- longitude
- latitude
- housing_median_age
- total_rooms
- total_bedrooms
- population
- households
- median_income
- ocean_proximity (categorical)

Target:

- median_house_value

---

3. AWS Architecture

S3 (Raw Data) ↓ Glue Training Job (Spark ML) ↓ Model Saved to S3 ↓ Glue Inference Job ↓ Predictions Saved to S3

Components:

- S3 Bucket: ml-capstone-project
- Glue Jobs: training + inference
- Glue Workflow: orchestration
- IAM Role: GlueMLCapstoneRole

---

4. Complete AWS Setup Steps

Step 1: Create S3 Bucket

1. Go to AWS Console → S3
2. Create bucket: ml-capstone-project
3. Create folders:
4. raw-data/
5. model/
6. predictions/
7. Upload housing.csv into raw-data/

Step 2: Create IAM Role

1. IAM → Create Role
2. Select AWS Service → Glue
3. Attach policies:
4. AWSGlueServiceRole
5. AmazonS3FullAccess
6. Role Name: GlueMLCapstoneRole

---

5. Glue Training Job

Create Job

- Name: housing-training-job
- Type: Spark
- IAM Role: GlueMLCapstoneRole

- Upload below script

Training Script (PySpark – Glue)

```
import sys
from pyspark.context import SparkContext
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.regression import RandomForestRegressor

spark = SparkSession.builder.appName("HousingTraining").getOrCreate()

# Read dataset from S3
df = spark.read.option("header", True).option("inferSchema", True)
    .csv("s3://ml-capstone-project/raw-data/housing.csv")

# Handle categorical column
indexer = StringIndexer(inputCol="ocean_proximity", outputCol="ocean_index")
encoder = OneHotEncoder(inputCol="ocean_index", outputCol="ocean_vec")

# Feature columns
feature_cols = [
    "longitude",
    "latitude",
    "housing_median_age",
    "total_rooms",
    "total_bedrooms",
    "population",
    "households",
    "median_income",
    "ocean_vec"
]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

# Model
rf = RandomForestRegressor(
    featuresCol="features",
    labelCol="median_house_value",
    numTrees=50
)

pipeline = Pipeline(stages=[indexer, encoder, assembler, rf])
```

```
model = pipeline.fit(df)

# Save model to S3
model.write().overwrite().save("s3://ml-capstone-project/model/")

spark.stop()
```

After execution:

- Model artifacts stored in S3
- Training completed successfully

---

6. Glue Inference Job

Create Job

- Name: housing-inference-job
- Type: Spark
- IAM Role: GlueMLCapstoneRole
- Upload below script

Inference Script (Static Data – No New Dataset Read)

```
from pyspark.sql import SparkSession
from pyspark.ml import PipelineModel

spark = SparkSession.builder.appName("HousingInference").getOrCreate()

# Load trained model
model = PipelineModel.load("s3://ml-capstone-project/model/")

# Define static input data
sample_data = [
    (-122.23, 37.88, 41, 880, 129, 322, 126, 8.3252, "NEAR BAY"),
    (-122.22, 37.86, 21, 7099, 1106, 2401, 1138, 8.3014, "NEAR BAY")
]

columns = [
    "longitude",
    "latitude",
    "housing_median_age",
    "total_rooms",
```

```
    "total_bedrooms",
    "population",
    "households",
    "median_income",
    "ocean_proximity"
]

static_df = spark.createDataFrame(sample_data, columns)

# Generate predictions
predictions = model.transform(static_df)

# Save predictions
predictions.select("prediction")
    .write.mode("overwrite")
    .csv("s3://ml-capstone-project/predictions/")

spark.stop()
```

After execution:

- Predictions saved in S3
- Model successfully deployed

---

7. Glue Workflow (Orchestration)

1. Glue → Workflows → Create Workflow
2. Name: housing-ml-workflow
3. Add Nodes:
4. Node 1: housing-training-job
5. Node 2: housing-inference-job
6. Set trigger:
7. Inference runs after Training succeeds

Now entire pipeline runs automatically.

---

8. End-to-End Execution Flow

1. Upload dataset to S3
2. Trigger Glue Workflow
3. Training Job executes
4. Model saved to S3
5. Inference Job runs automatically

6. Predictions saved to S3
7. Pipeline completes

This demonstrates a production-ready ML orchestration system.

---

9. Where AI Comes Into This Project

Artificial Intelligence (AI) is the broader concept of building intelligent systems.

Machine Learning (ML) is a subset of AI that enables systems to learn patterns from data.

In this project:

- The model learns relationships between housing features and prices
- It generalizes from historical data
- It predicts unseen values
- It makes automated decisions

Thus, this solution is an AI-enabled predictive system implemented using ML engineering on AWS.

---

10. Technical Highlights

- Distributed training using Spark
- Feature engineering (StringIndexer + OneHotEncoder)
- RandomForestRegressor model
- Model persistence in S3
- Static inference deployment
- Automated orchestration via Glue Workflow

---

11. Summary

Designed and implemented an end-to-end AI/ML pipeline on AWS using Glue and Spark. Built automated orchestration for training and inference, stored models in Amazon S3, and deployed a scalable housing price prediction system using distributed ML.